Now, RSpec's output shows both failures underneath a description of the example:

```
$ bundle exec rspec spec/integration/app/ledger_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::Ledger#record with a valid expense successfully saves
  the expense in the DB
     Got 1 failure and 1 other error:

     1.1) Failure/Error: expect(result).to be_success
            expected nil to respond to `success?`
          # ./spec/integration/app/ledger_spec.rb:23:in `block (4 levels)
          in <module:ExpenseTracker>'

     1.2) Failure/Error: id: result.expense_id,

          NoMethodError:
            undefined method `expense_id' for nil:NilClass
          # ./spec/integration/app/ledger_spec.rb:25:in `block (4 levels)
          in <module:ExpenseTracker>'

Finished in 0.02142 seconds (files took 0.15952 seconds to load)
1 example, 1 failure

Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:20 #
ExpenseTracker::Ledger#record with a valid expense successfully saves the
expense in the DB

Randomized with seed 41929
```

We're likely to want our other integration tests to get this benefit. Move the :aggregate_failures property up one level from the example to the group:

06-integration-specs/05/expense_tracker/spec/integration/app/ledger_spec.rb

```
RSpec.describe Ledger, :aggregate_failures do
```

RSpec refers to these properties of specs as *metadata*. When you define metadata—arbitrary symbols or hashes—on an example group, all the examples in that group inherit it, as do any nested groups. We previously saw an example of metadata being specified as a hash in *Tag Filtering*, on page 25. Here we are defining our metadata using just a symbol as a shortcut. Internally, RSpec expands this into a hash like { aggregate_failures: true }.

Now, it's time to fill in the behavior. The Ledger class's record method needs to store the expense in the database, then return a RecordResult indicating what happened:

06-integration-specs/06/expense_tracker/app/ledger.rb

```
def record(expense)
```